Name: Vein Carmell G. Pangilinan
Course and Section: IT342 - G2
Project Name: StillNess
Date: March 9, 2026

Phase 1: Web User Registration and Login

GitHub Repository Link

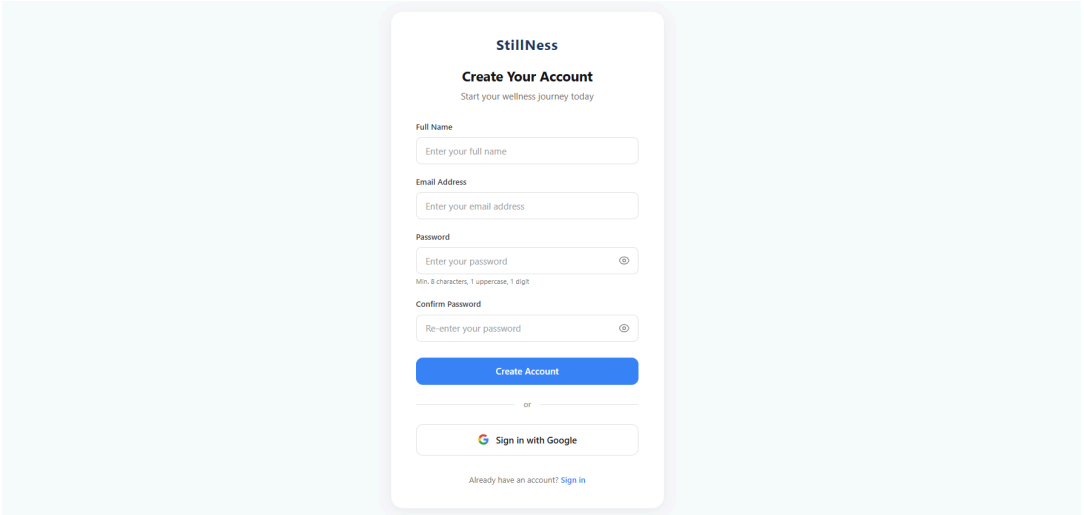
Repository URL	https://github.com/veinP/IT342-Pangilinan-Stillness.git
Repository Name	IT342-Pangilinan-Stillness
Backend Group ID	edu.cit.pangilinan
Backend Artifact ID	stillness
Base Package	edu.cit.pangilinan.stillness
Framework	Spring Boot 3.5.x | Maven | REST API

Final Commit for Phase 1

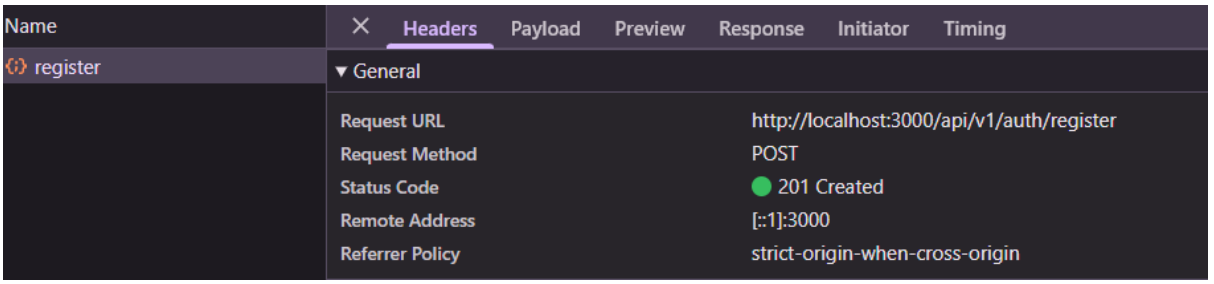
Commit Message:	IT342 Phase 1 – User Registration and Login Completed
Commit Hash	11ec1418d0e9e2424a6f2619bb40bada942d4b93
Commit Link	https://github.com/veinP/IT342-Pangilinan-Stillness/commit/11ec1418d0e9e2424a6f2619bb40bada942d4b93

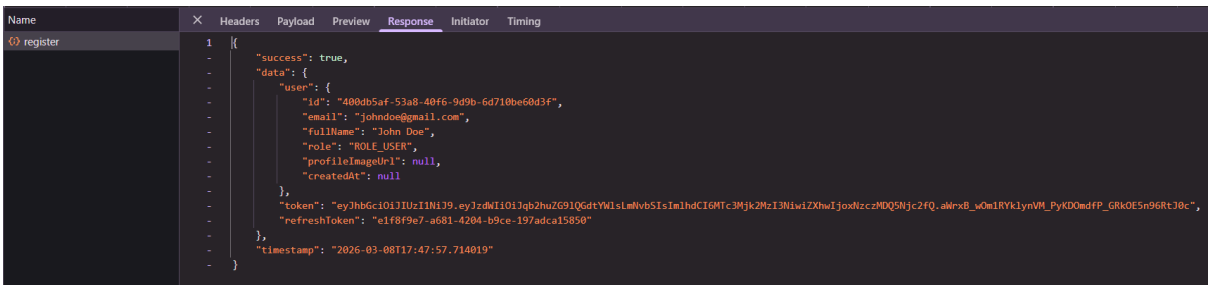
Screenshots

1. Registration Page

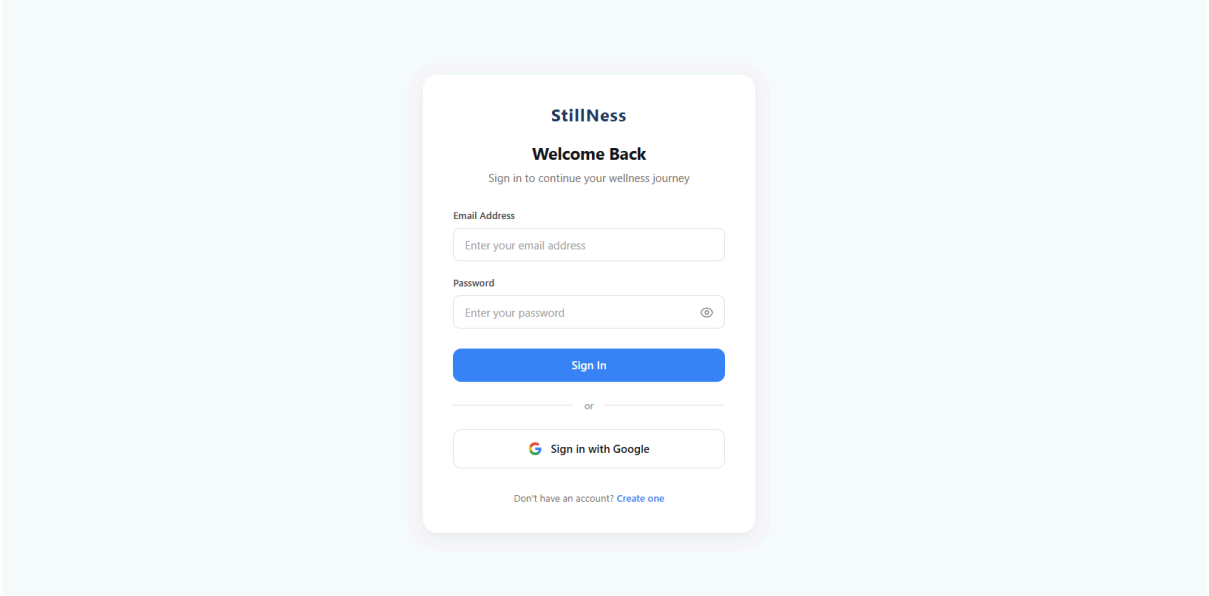


2. Successful User Registration

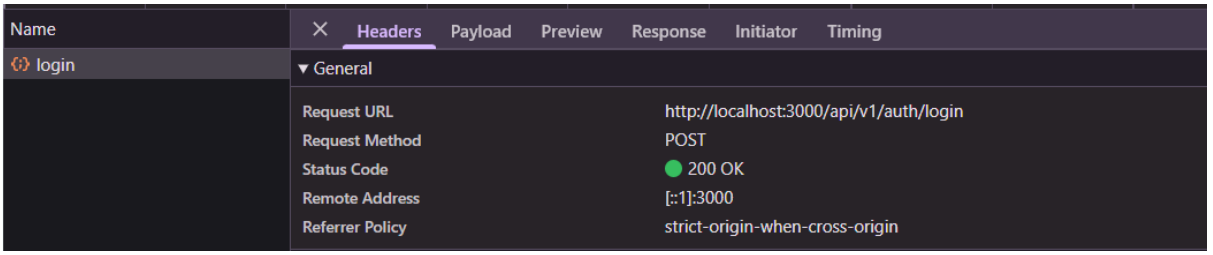


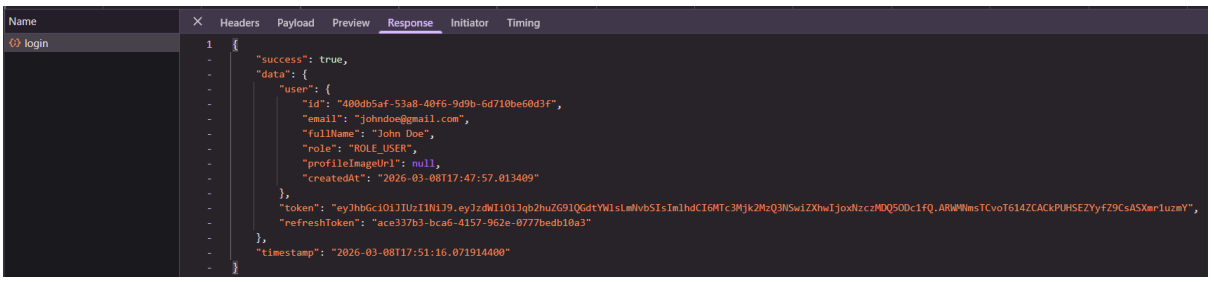


3. Login Page

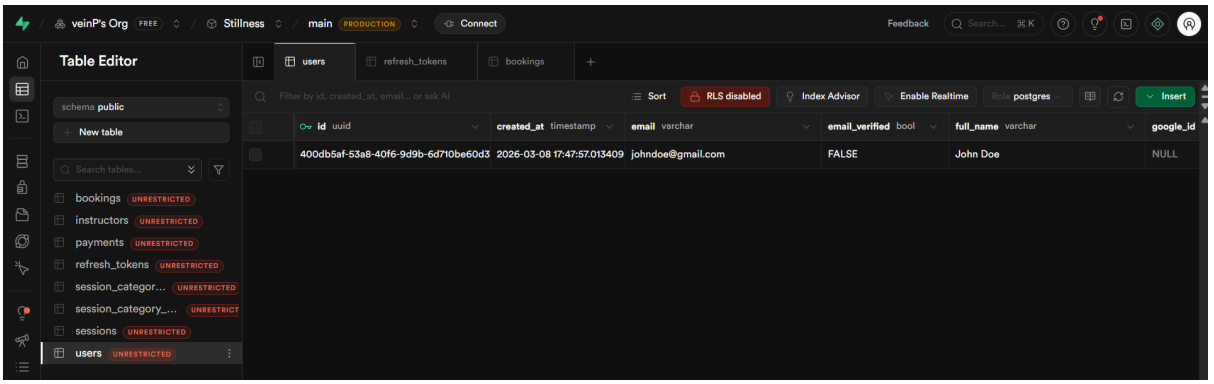


4. Successful Login





5. Database Record (Supabase)



Short Implementation Summary

1. User Registration

Registration Fields

Field	Type	Validation Rules
fullname	String	Required — must not be blank (@NotBlank)
email	String	Required — valid email format (@Email)
password	String	Required — min 8 chars, ≥1 uppercase, ≥1 digit (@Pattern: ^(?=.*[A-Z])(?=.*\d).{8,}$)
confirmPassword	String	Required — must exactly match the password field

Validation Process

All incoming registration requests are validated using Jakarta Bean Validation annotations (@NotBlank, @Email, @Pattern). The password is validated against the regex ^(?=.*[A-Z])(?=.*\d).{8,}$. If the password and confirmPassword fields do not match, the server returns a 400 Bad Request with error code VALID-001.

Duplicate Account Prevention

Before creating a new user, the system calls userRepository.existsByEmail(email) to check for an existing record. If a match is found, a 409 Conflict response is returned with error code DB-002 ("Email already registered"). The email column also carries a UNIQUE database constraint as a secondary safeguard.

Secure Password Storage

Passwords are hashed using BCrypt with a strength factor of 12 via Spring Security's BCryptPasswordEncoder bean before being written to the password_hash column. Plain-text passwords are never persisted at any point in the pipeline.

2. User Login

Login Credentials

Field	Validation Rules
email	Required — must be a valid email format
password	Required

Credential Verification Flow

a. The controller passes credentials to Spring Security's Authenticate Manager via UsernamePasswordAuthenticationToken.
b. The DaoAuthenticationProvider loads the user by email and compares the submitted password against the stored BCrypt hash using BCryptPasswordEncoder.matches().
c. If credentials are invalid, a 401 Unauthorized response is returned with error code AUTH-001.

After Successful Login

a. The user's last_login_at timestamp is updated in the database.
b. A JWT access token is generated (HS256, signed with a secret key, subject = user email).
c. A refresh token (UUID-based) is created and stored in the refresh_tokens table.

d. An AuthResponse is returned containing the user object (id, email, fullName, role, profileImageUrl, createdAt), the JWT access token, and the refresh token.

Database Tables

Table: users

Column	Type	Constraints / Notes
id	UUID	Primary Key — auto-generated
email	VARCHAR	NOT NULL, UNIQUE
password_hash	VARCHAR	Stores BCrypt hash (never plain-text)
full_name	VARCHAR	User's display name
role	VARCHAR(50)	ROLE_USER or ROLE_ADMIN
profile_image_url	VARCHAR	Optional profile picture URL (nullable)
google_id	VARCHAR	OAuth2 Google identifier (nullable)
email_verified	BOOLEAN	Email verification flag
created_at	TIMESTAMP	Auto-set on account creation
updated_at	TIMESTAMP	Auto-set on record update
last_login_at	TIMESTAMP	Updated on every successful login

Table: refresh_tokens

Column	Type	Constraints / Notes
id	UUID	Primary Key
user_id	UUID	Foreign Key -> users.id
token	VARCHAR(512)	NOT NULL, UNIQUE
expiry_date	TIMESTAMP	Token expiration date/time
device_info	VARCHAR	Optional device identifier (nullable)
revoked	BOOLEAN	NOT NULL, default false
created_at	TIMESTAMP	Auto-set on creation

API Endpoints
Base path: /api/v1

Method	Endpoint	Description	Auth
POST	/api/v1/auth/register	Register a new user account	No
POST	/api/v1/auth/login	Authenticate and receive JWT + refreshToken	No
POST	/api/v1/auth/logout	Revoke all refresh tokens for the user	JWT
GET	/api/v1/auth/me	Get current authenticated user info	JWT